RL-Course 2024/25: Final Project Report

Reinforce the Puck: Jonathan Schwab, Tom Freudenmann

February 25, 2025

1 Introduction

This project focuses on the development and evaluation of Twin Delayed Deep Deterministic Policy Gradient (TD3), Soft Actor Critic (SAC), and a hybrid model that combines both, designed for a competitive hockey environment. The goal is to develop these agents from scratch and optimize their performance with tailored modifications, including improved replay buffers, reward shaping, exploration noise strategies, and self-play.
The hockey environment requires precise control and strategic decision-making in a continuous state and action space. We designed a PyTorch-based framework supporting multiple environments, enabling us to train, fine-tune, and evaluate our agents on different tasks.
Our implementation follows the original research papers with modifications to enhance training stability and efficiency. We introduced Balanced Experience Replay (BER) and Prioritized Experience Replay (PER) to improve sample efficiency and used pink noise for better exploration.
To further enhance performance, we developed a hybrid model that selects between TD3 and SAC using a Double Deep Q-Network (DDQN) gate, dynamically choosing the best agent per state.
We first evaluated our models on standard benchmarks (Pendulum, Lunar Lander, and Half Cheetah) before training them in the hockey environment. This report details our methodology, experimental setup, and results.

2 Method

This chapter describes the methodological framework utilized in developing and evaluating reinforcement learning agents for the hockey environment. We introduce first TD3 (2.1)[1] and explain our key changes to enhance the learning behavior and final reward with pink noise (2.2)[1], new replay buffers (2.3)[1] and our custom reward design (2.4)[1]. Afterward, SAC (2.5)[2] and the hybrid model (2.6)[2] are introduced in detail.

2.1 Twin Delayed Deep Deterministic Policy Gradient

Our TD3 algorithm is a from scratch implementation of the original paper [2]. TD3 is an off-policy actor-critic algorithm for continuous action and state spaces. This combination is very vulnerable to overestimation bias and error accumulation in Temporal Difference (TD)-learning [2]. The overestimation bias is related to discrete Q-learning, where the value estimate is updated with the following greedy target:

[1] Tom Freudenmann
[2] Jonathan Schwab

$$y = r + \gamma \max_{a'} Q(s', a') \leq \mathbb{E}_{\epsilon} \left[max_{a'}(Q(s', a') + \epsilon) \right] \tag{1}$$

Where r is the reward, γ the discount factor, s' the next state, and a' the next action. Thus, the result is generally larger than the true maximum for the estimate with the error ϵ [12]. In actor-critic environments, the policy is updated with a deterministic policy gradient, which leads to suboptimal updates and diverging behavior in the learning process [2].
Therefore, TD3 introduces clipped double Q-Learning that is based on DDQN. DDQN optimizes two actors with two critics, which use the same replay buffer and the opposite critic in the learning process [14], inducing an overestimation of some states. In contrast, TD3 uses a single actor π_ϕ which is optimized with respect to the minimum of two critics $Q_{\theta_1}, Q_{\theta_2}$:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_\phi(s')) \tag{2}$$

As a result, the computational costs are reduced because a single actor is used. But instead, an underestimation bias is induced that is preferable to overestimation since it does not propagate through the policy updates [2].
Another challenge are the high variance estimates, causing a noisy policy gradient. Thus, the Bellman equation is never exactly satisfied, resulting in a small residual TD-error δ_t. This error propagates through the update step, such that the expected return depends on the expected discounted difference between reward and TD-error [2]:

$$Q_\theta(s_t, a_t) = \mathbb{E}_{s_i \sim p_\pi, a_i \sim \pi} \left[\sum_{i=t}^{T} \gamma^{i-t} (r_i - \delta_i) \right] \tag{3}$$

TD3 addresses this problem by using slowly changing target networks for actors and critics and introducing policy smoothing regularization. Firstly, target networks are used to reduce the error over multiple updates by providing a stable learning objective and reducing the variance in the policy updates [2]. In TD3 the target network's parameters θ are updated with a small rate τ:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \tag{4}$$

Secondly, target policy smoothing regularization is introduced to reduce the vulnerability to narrow peaks in the value estimate. The main idea is that similar actions in the continuous space should have similar values, which is related to the learning update from State-action-reward-state-action (SARSA) algorithm [11]. To enforce the relationship between similar actions, a small area around the target action is fitted by adding a small amount of Gaussian clipped noise. This results in the following update step:

$$y = r + \gamma Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon) \quad |\epsilon \sim clip(\mathcal{N}(0, \sigma), -c, c) \tag{5}$$

Moreover, TD3 extends the basic Deep Deterministic Policy Gradient (DDPG) algorithm [6] by resolving the overestimation bias, adding soft target network updates, and smoothing the target policy to further lower the value estimate variance. As a result, it is a state-of-the-art actor-critic algorithm, which shows an improvement in learning speed and performance compared to DDPG in some tasks in continuous domains. We further adapt the proposed implementation to the hockey environment by using different replay buffers (subsection 2.3), reward functions (subsection 2.4) and pink noise (subsection 2.2) as exploration noise.

2.2 Pink Noise

Colored noise as exploration noise for off-policy algorithms was introduced by [1]. It aims to balance out the exploitation-exploitation trade-off. In general, white noise $\mathcal{N}(0, \sigma)$ enforces less exploration and more exploitation, while strong correlated noise like Ornstein-Uhlenbeck focuses on exploring hard actions, resulting in following off-policy trajectories [1]. Colored Noise introduces a new hyperparameter β that defines the noise color. Therefore, white noise is sampled and transformed by the fast Fourier transform into the frequency domain. The frequencies f are scaled by $f' = f^{-\beta/2}$ and transformed back into the time domain. We use pink noise ($\beta = 1$) because it enhances exploration and exploitation in complex tasks [1].

2.3 New Replay Buffers

The basic approach for modeling experience buffers are Experience Replay (ER) buffers [7], which hold the past n (most of the time $n > 100000$) transitions. At training time, m transition batches are sampled to train actor and critic networks. Although each iteration is sampled uniformly, early observations in the empty buffer are drawn more often than late experiences [9]. Instead, we propose first BER [10] and PER [9] to ensure a balanced or prioritized sampling of late and new experiences.

Balanced Experience Replay BER [10] is a balanced version of the default ER where each experience gets a corresponding memory strength. This strength is reduced every time a new experience is added to the buffer. The resulting probability to draw an experience j from the buffer is $P(j) = \frac{s_j^\alpha}{\sum_i s_i^\alpha}$, where s_i corresponds to the strength of experience i [10].

Prioritized Experience Replay PER is introduced by [9] and focuses on prioritizing experiences with a big TD-error. Thus, each experience j has a sampling probability $P(j) = \frac{\delta_j^\alpha}{\sum_i \delta_i^\alpha}$, where δ_i is the TD-error of experience i and α is a hyperparameter describing the influence of the priorities. In general, this prioritization leads to a bias concerning the sampled data. This bias is removed by multiplying the update weights by the probability of drawing [9]. To further balance the sampling, we implement the buffer as Sum Trees, which allow sampling and adding data in $O(log\, n)$ [9].

2.4 Custom Reward Design

The hockey environment already provides a set of reward signals, including a positive signal for winning, the distance to the ball, contact to the ball, and the direction of the pucks' movement. As we observed, our early state agents have problems playing the puck when it starts on their side. Therefore, we added a time penalty, which decreases the overall reward for the complete run, and a penalty for not touching the not moving puck after 10% of the environment steps.
Additionally, TD3 has an issue with aggressive game play. Thus, we added reward for touching the ball when it is moving with negative x-velocity. To further force the agent to minimize its distance to the goal center, we added a linear reward for small distances. To combine all signals, we added weights w_i for all reward signals r_i and provided the agent with the weighted sum over all signals $R = \sum_{i=1}^{8} w_i \cdot r_i$.

2.5 Soft Actor Critic

Our SAC implementation is based on the original paper by [4] and its follow-up paper [5]. SAC is an off-policy actor-critic algorithm that learns a stochastic policy in an environment with continuous action and state spaces. The algorithm is based on the principle of maximum entropy reinforcement learning, which

aims to maximize the expected reward while also maximizing the entropy of the policy. This encourages exploration and prevents the policy from getting stuck in local optima. The major components of the SAC algorithm are the actor, the critic and the entropy target.

The actor is a stochastic policy network, which is based on a vanilla neural network with a Gaussian distribution output layer. It estimates the mean and standard deviation $\mu(s), \sigma(s) = f_\theta(s)$ and provides a sampling method that returns both a sample from the distribution and the log probability of the selected sample. To make the sampling process differentiable, the reparameterization trick is used where $z \sim \mathcal{N}(0, 1)$ and $u = \mu(s) + \sigma(s) \cdot z$. The action bounds are enforced by applying the $\tanh$ function to the sampled unbounded action. To account this bounding in the probability density function, the probability density function of the unbounded action is scaled by the absolute determinant of the Jacobian matrix of the $\tanh$ function [2][C. Enforcing Action Bounds]:

$$\pi(a \mid s) = \mu(u \mid s) \left| \det \left(\frac{d\mathbf{a}}{d\mathbf{u}} \right) \right|^{-1}. \tag{6}$$

For each state, sampled from the replay buffer $\mathcal{D}$ an action is sampled from the actor, and the log probability for this action is calculated using the adjusted probability density function. The actor is trained on the maximum entropy objective, where the critic estimates the value for each state-action pair:

$$J(\pi) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi} \left[\alpha \log \pi(a_t \mid s_t) - Q(s_t, a_t) \right]. \tag{7}$$

The entrophy term $\log \pi(a_t|s_t)$ is weighted by the temperature parameter α to balance exploration and exploitation. The critic consists of two Q-networks, valuing state-action pairs. During training, both networks independently predict a value for the given state-action pair ($s_t \sim \mathcal{D}, a_t \sim \pi$) and the minimum is selected. This is done to prevent overestimation bias [3].

The target Q-values incorporate the immediate reward r and the discounted future reward. Additionally, the weighted entropy term is included. This formulation aligns with the objective function $J(\pi)$, which maximizes both the expected reward and the entropy of the policy:

$$y = r + \gamma(1 - d) \left(\min \left(Q_1^{\text{target}}(s', a'), Q_2^{\text{target}}(s', a') \right) + \alpha \mathcal{H}(\pi(\cdot \mid s')) \right). \tag{8}$$

The critic is trained to optimize the mean squared Bellman error, which represents the difference between the estimated Q-values and the target values:

$$\mathcal{L}_Q = \mathbb{E} \left[(Q(s, a) - y)^2 \right]. \tag{9}$$

To stabilize the training process, the target Q-networks are updated using a soft target update mechanism, as it was shown to improve the stability of the training process [4][E. Additional Baseline Results].

In the original paper "an additional function approximator for the value function [was introduced] but later found [as] to be unnecessary" [5], so we omitted it.

Moreover, the automatic temperature parameter tuning using gradient-based optimization, introduced in the follow-up paper [5], was implemented:

$$J(\alpha) = \mathbb{E} a_t \sim \pi \left[-\alpha \left(\log \pi(a_t \mid s_t) + \mathcal{H}\text{target} \right) \right], \tag{10}$$

where $\mathcal{H}$target is set to negative of the action space dimensionality, as proposed in [5][D. Hyperparameters].

2.6 Hybrid-Model

While reviewing the gameplay of the trained SAC and TD3 agents, we noticed that both agents have their strengths and weaknesses in different situations. To combine the strengths of both agents, we created a hybrid model. This approach is inspired by the Mixture of Experts (MoE) architecture, where multiple experts are combined using a gating network to weight the experts predictions. But since we've got a continuous spatial action space, weighting both experts predictions would not be a feasible option. Thus, a DDQN [13] was implemented as a selection gate for subagents. The subagents are encoded as discrete numbers (actions). This gate estimates the value of choosing a specific subagent, given a state:

$$i^*(s) = \arg \max_{i \in \text{Sub-Agents}} Q(i, s) \tag{11}$$

$$a = \pi_{i^*(s)}(s) \tag{12}$$

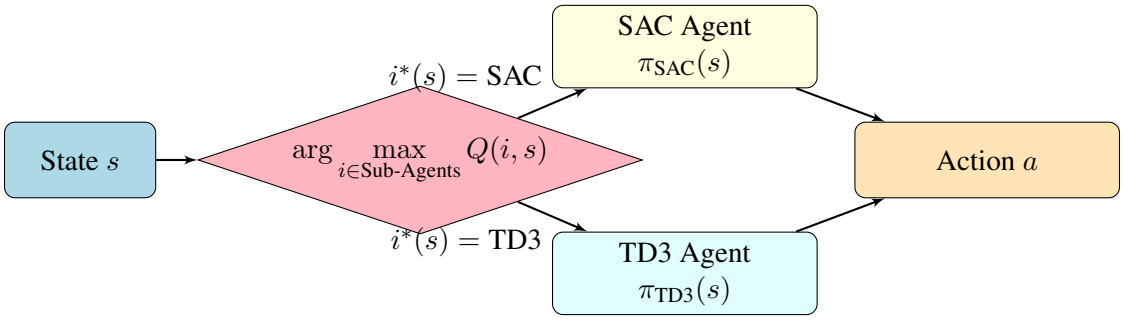

Figure 1: Hybrid model with DDQN-based agent gate using SAC and TD3 as sub-agents.

The loss function for training the DDQN is based on the TD-Error, which is the difference between the predicted and the target Q-value. We investigated three different training strategies for the hybrid model:

- Training from scratch: Training the subagents and the gating network simultaneously from scratch.
- Training by fine-tuning the sub-agents: Using sub-agents that are pre-trained independently and further fine-tuned simultaneously to the gating network.
- Training with fixed sub agents: Using subagents that are pre-trained independently and left fixed while training the gating network.

3 Evaluation[3]

With all our methodologies in place, we now investigate their performance in different domains. We start with analyzing the performance on 3 simple environments (3.1) to show the impact of our modifications at well-known reinforcement learning benchmarks. Afterward, we focus on the hockey environment, where the training of the foundation models is analyzed and compared to the results of the simple environments (3.2). Finally, the hybrid model is evaluated on the hockey environment and shortly compared to the other implementations (3.3).

[3]Data recording performed equally. Text: TD3 + modifications (Tom Freudenmann), SAC + hypbrid (Jonathan Schwab)

3.1 Training on Simple Environments

We trained the introduced algorithms first on simple environments, also used in the lecture. Therefore, we compare TD3 with the different buffers and basic SAC on Pendulum, Lunar Lander, and Half Cheetah. In this step, we exclude our hybrid approach because it was specifically developed for the hockey environment. With the attached hyperparameters (Appendix B.1), we can observe that SAC has higher average returns after 10000 steps than all TD3 implementations, and TD3 with BER outperforms the other buffer implementations in Half Cheetah and Lunar Lander (Table 2). Additionally, PER always performs better than ER. With these results, we expect equal behavior in the hockey environment.

Table 1: Average Reward for Simple Environments and Algorithm (Last 100 Steps)

Environment	SAC (ER)	TD3 (ER)	TD3 (PER)	TD3 (BER)
HalfCheetah	**3296.86**	1739.69	1874.47	1999.50
Lunar Lander	**143.82**	-41.53	-22.13	61.10
Pendulum	-249.59	-163.91	**-145.48**	-181.35

3.2 Hockey Environment

The performance on the hockey environment is evaluated with the average final performance (Table 2) and the corresponding evaluation reward of the training in 600000 environment steps (Figure 2). Therefore, we train SAC and TD3 on the original environment reward and our own reward. The opponents are the strong/weak basic opponents, SAC foundation, TD3 foundation, and Hybrid foundation agents, which are pretrained on the strong/weak opponents. To stabilize training, the opponents change after 10000 environment steps (100 epochs). The evaluation reward contains only the win/lose signal with $r_{eval} \in \{-10, 0, 10\}$ and is recorded every 5000 steps (50 epochs) over 20 runs. The average reward is then saved and plotted in Figure 2. All other statistics, like critic-, actor-loss and training reward, are provided in the appendix subsection B.5. The final average evaluation reward over 100 runs is also provided in Table 2.

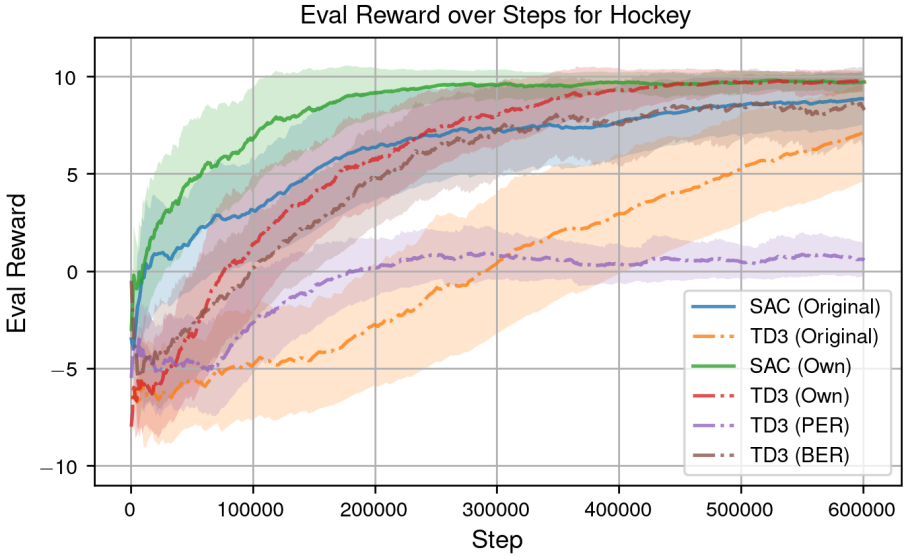

Figure 2: Evaluation Rewards for SAC and TD3 with original and own reward.

Table 2: Average Evaluation Reward for Hockey Environment

Environment	SAC	TD3	TD3 (White)	TD3 (Blue)	TD3 (PER)	TD3 (BER)
Hockey Original Reward	**8.95**	7.16	-	-	-	-
Hockey Own Reward	9.70	**9.71**	8.56	9.36	0.68	8.22

TD3 is evaluated for both rewards, different exploration noises, and the presented buffer types, with pink noise and ER as default configuration. Firstly, we see that the behavior of the buffers behaves differently in simple environments. As a result, the PER buffers lead to a suboptimal policy, which results in a tie for the most runs. The graphical evaluation showed that the agent learns to shoot the puck against the boarder, such that it bounces the complete run over the vertical center line. Furthermore, the critic-loss is the lowest of all TD3 algorithms, such that we assume that the critic overfits to states with initial high TD-error. Besides, BER has a higher variance than ER in the evaluation, indicating that the buffer is more uncertain in its prediction. This leads to a lower overall performance compared to ER.
Secondly, white ($\beta = 0$) and blue ($\beta = 0.5$) noise learn slower than the agent with pink noise and have a lower final evaluation reward. Thus, the results of [1] also apply to the hockey environment by well balancing the exploitation-exploration trade-off.
Finally, TD3 in particular benefits from the proposed new reward, containing a time penalty, the distance to the goal center, and the refusal to start penalty. Thus, TD3 with the new reward converges after ~ 40000 environment steps to an almost perfect policy for the proposed opponents, while TD3 with the original reward converges not in our experiment. Furthermore, the new reward helps TD3 to beat SAC and increases the final average evaluation reward by more than 2.
Instead, SAC performs already well on the default reward, but converges faster with the custom reward. In Figure 2 we can see that our custom reward function almost halves the number of steps required for convergence compared to the standard reward function. Furthermore, the final evaluation reward only increased by 0.8 compared with the custom reward. Another interesting aspect is that SAC converges faster compared to TD3, which can be explained by the implemented alpha-tuning.
Finally, both algorithms beat the weak opponent with 99.20% (TD3) and 98.40% (SAC). Instead, against the strong opponent, they win with a rate of 97.90% (TD3) and 97.10% (SAC) (appendix B.8).

3.3 Hybrid Model Evaluation

The hybrid model is trained against the basic strong and weak opponent, as well as foundation and fine-tuned SAC and TD3 checkpoints. To prevent overfitting and enhance generalization, opponents are periodically changed throughout training. SAC and TD3 foundation checkpoints also serve as submodels in the setup with pretrained experts. As shown in Figure 3, the training strategy that combines pretrained submodels with simultaneous training of the gate and submodels achieves the highest evaluation reward. The strategy that starts with empty submodels and trains both the gate and submodels also performs well but fails to converge to the maximum reward of ten. Meanwhile, the approach where only the gate is trained initially yields the best evaluation reward but is eventually outperformed by the other strategies as training progresses. The gate's loss reflects its effectiveness in selecting the optimal submodel for a given state. In Figure 3, periodic peaks in the loss indicate moments when opponents were changed. The training strategy with pretrained submodels and simultaneous gate-submodel training results in the smoothest loss curve, with a steady decrease after 60,000 steps. In contrast, other strategies exhibit high fluctuations, and their average loss tends to increase over time. Finally, our best Hybrid model wins with 97.70% and 94.60% against the weak and strong opponents (appendix B.8).

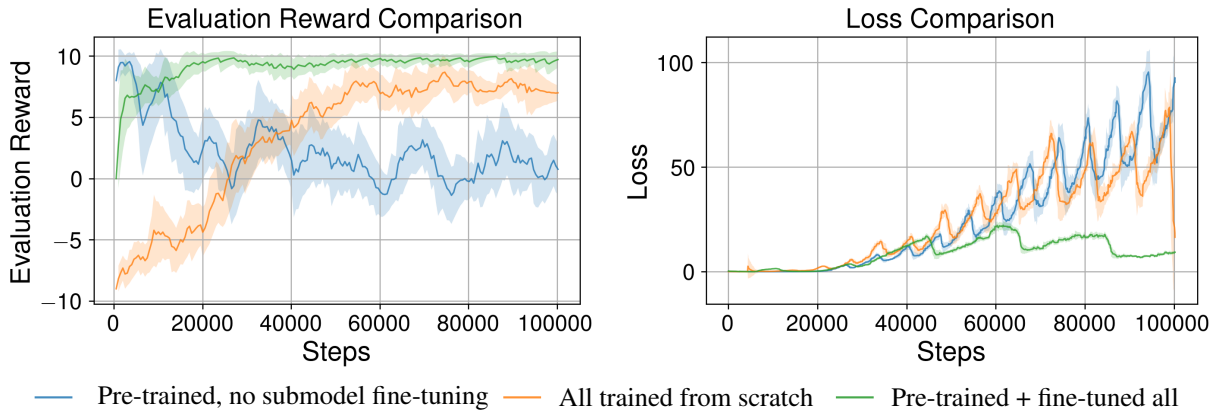

Figure 3: Evaluation reward and loss comparison of hybrid model training strategies

4 Discussion

In this project, we implemented a strongly modular framework for basic gymnasium environments, which can be easily extended to other environments like Hockey. This enabled us to provide a deep evaluation based on multiple environments and for multiple hyperparameter configurations. Furthermore, the framework allows us to implement and compare multiple agents with each other based on the implemented tournament (**??**). The presented evaluation results, show how big the influence of better buffers, rewards, exploration noise on different model types and understanding the game can be.

Firstly, we see that prioritization or balancing of buffers works well on well-known benchmarks, but cannot be applied to the Hockey environment. This is strongly related to the sparse reward and big buffers required for the environment. Instead, the exploration with pink noise is a good choice, because we observe the highest final reward and fastest convergence in our evaluation.

Furthermore, SAC is shown to converge almost in half of the steps compared to our best TD3. This effect is enforced by the alpha-tuning, which dynamically weights the exploration noise. Together with our custom reward containing penalties for aggressive behavior and refusal to play, we can provide almost equal performing SAC and TD3 implementations.

Moreover, our hybrid model is highly adaptable and based on the previously trained agents, with the aim of compensating for the weaknesses of the other model. Thus, it can use the more aggressive TD3 agent for shooting and the passive SAC for defending the goal. However, we saw in our training and evaluation experiments that it tends to overfit fast to the provided opponents.

In addition, the graphical evaluation shows that SAC and TD3 learn different policies. SAC tends to shoot directly to the goal and act more passive. Instead, focuses TD3 on shooting over the horizontal walls by being more aggressive in the center of the field.

Besides, we saw that especially TD3 and SAC are unstable in the simultaneous self-play, where all agents are trained simultaneously (subsection B.6). Instead, the training against previously trained fixed checkpoints as self-play is more effective and used in the evaluation.

In conclusion, this report shows that the game factor of the hockey environment is more important than small algorithm modifications. The hockey environment tends to overfit to specific opponents, requiring numerous training runs and making performance heavily dependent on the choice of opponents. In the challenge, different opponents are used, such that a general policy is required to compete against the other teams. Thus, the biggest improvements are made through own rewards, fixed self-play, and long training runs, which help to train a general policy. Furthermore, our framework helps to explore more algorithms like CrossQ, hierarchical SAC, or new hyperparameter combinations through mutations.

References

[1] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.

[2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[3] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[5] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. 2019.

[6] T. Lillicrap. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[7] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8:293–321, 1992.

[8] R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 24(2):193–202, 1975.

[9] T. Schaul. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

[10] H. Sun, R. Li, H. Yang, and W. Zhu. Balanced prioritized experience replay. In *2022 3rd International Conference on Electronic Communication and Artificial Intelligence (IWECAI)*, pages 200–203, 2022.

[11] R. S. Sutton and A. G. Barto. Reinforcement learning: An introduction. *Robotica*, 17(2):229–235, 1999.

[12] S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school*, pages 255–263. Psychology Press, 2014.

[13] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning, 2015.

[14] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In M. F. Balcan and K. Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, New York, New York, USA, 20–22 Jun 2016. PMLR.

A Acronyms

BER Balanced Experience Replay

DDPG Deep Deterministic Policy Gradient

DDQN Double Deep Q-Network

ER Experience Replay

MoE Mixture of Experts

PER Prioritized Experience Replay

SAC Soft Actor Critic

SARSA State-action-reward-state-action

TD Temporal Difference

TD3 Twin Delayed Deep Deterministic Policy Gradient

B Additional Plots and Hyperparameter

This section contains plots and hyperparameter related to the evaluation. They are intended as supplementary material and emphasis the correctness of the evaluation.

B.1 Hyperparameter for simple evaluation

This section contains our hyperparameter configuration for Pendulum (Table 3), Lunar Lander (Table 4) and Half Cheetah (Table 5). Please note, that SAC has always the same hyperparameters. In contrast, TD3 requires different hyperparameter configurations to work better on the specified environments.

Table 3: Hyperparameter Summary for Pendulum

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	ER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	BER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	PER	0.95
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

Table 4: Hyperparameter Summary for LunarLander

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	ER	0.98
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	BER	0.98
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	PER	0.98
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

Table 5: Hyperparameter Summary for HalfCheetah

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	ER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	BER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	PER	0.95
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

B.2 Evaluation Plots for Pendulum

In this section, we present the evaluation plots for the Pendulum environment. The plots illustrate the performance of the TD3 and SAC algorithms over the environment steps. As observed, TD3 consistently outperforms SAC in terms of reward (Figure 4), actor loss (Figure 5) and critic loss (Figure 6). The following figures provide a detailed comparison of the two algorithms' performance metrics.

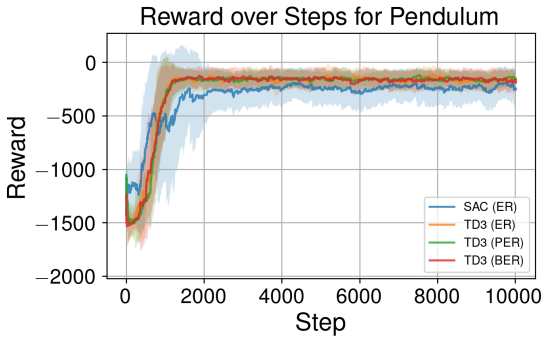

Figure 4: Reward over the environment steps

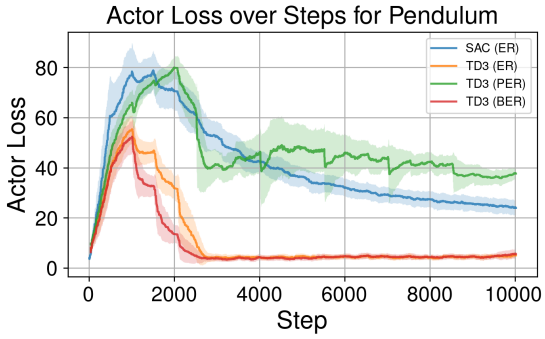

Figure 5: Actor Loss over the environment steps

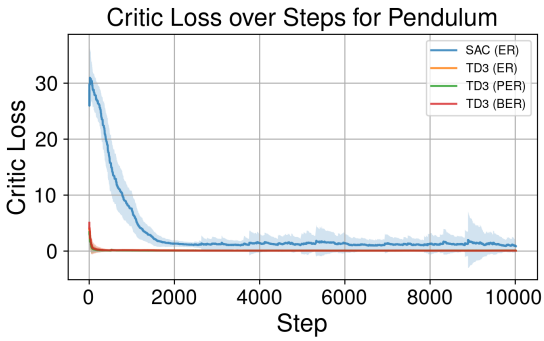

Figure 6: Critic Loss over the environment steps

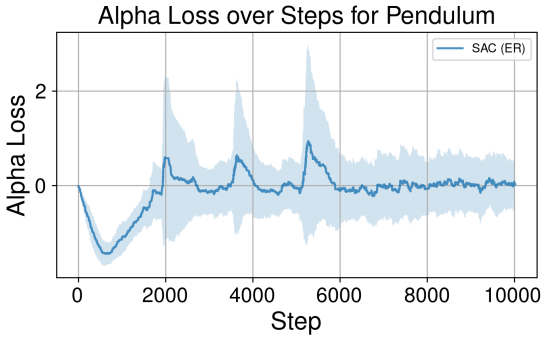

Figure 7: Alpha Loss over the environment steps

B.3 Evaluation Plots for Lunar Lander

In this section, we present the evaluation plots for the Lunar Lander environment. The plots illustrate the performance of the TD3 and SAC algorithms over the environment steps. As observed, SAC consistently outperforms all TD3 variants in terms of reward (Figure 8), actor loss (Figure 9) and critic loss (Figure 10). The following figures provide a detailed comparison of the two algorithms' performance metrics.

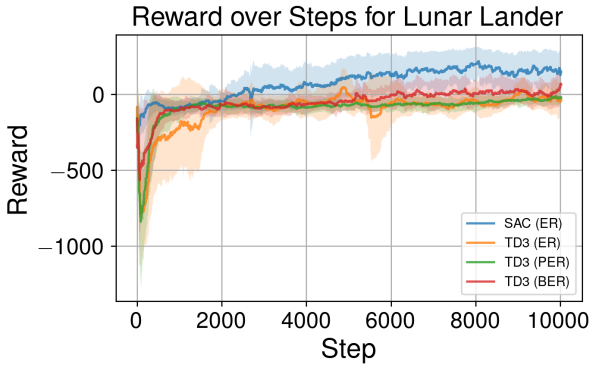

Figure 8: Reward over the environment steps

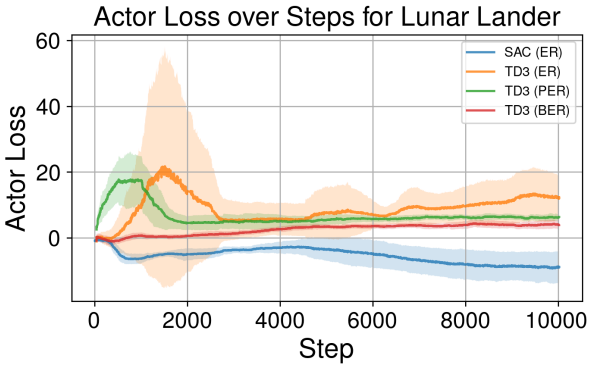

Figure 9: Actor Loss over the environment steps

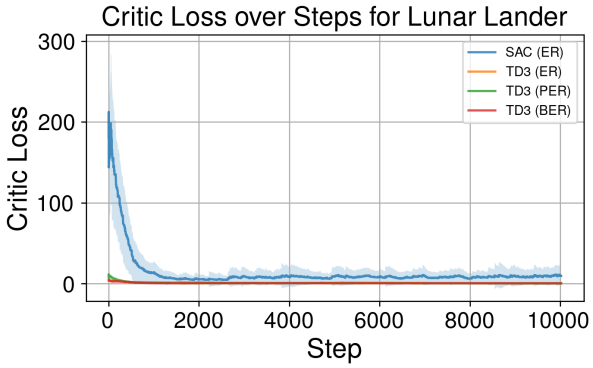

Figure 10: Critic Loss over the environment steps

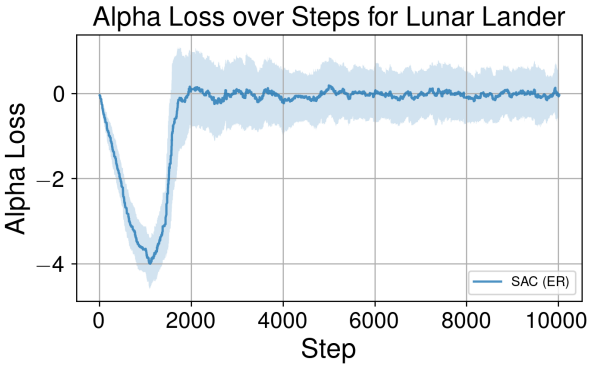

Figure 11: Alpha Loss over the environment steps

B.4 Evaluation Plots for HalfCheetah

In this section, we present the evaluation plots for the HalfCheetah environment. The plots illustrate the performance of the TD3 and SAC algorithms over the environment steps. As observed, SAC clearly outperforms all TD3 variants and converges much faster and to a higher reward (Figure 12). In contrast, TD3 learns only slowly. The following figures provide a detailed comparison of the two algorithms' performance metrics, including actor loss (Figure 13), critic loss (Figure 14), and alpha loss (Figure 15).

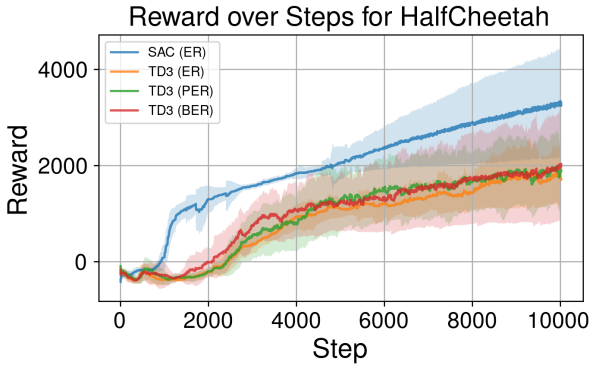

Figure 12: Reward over the environment steps

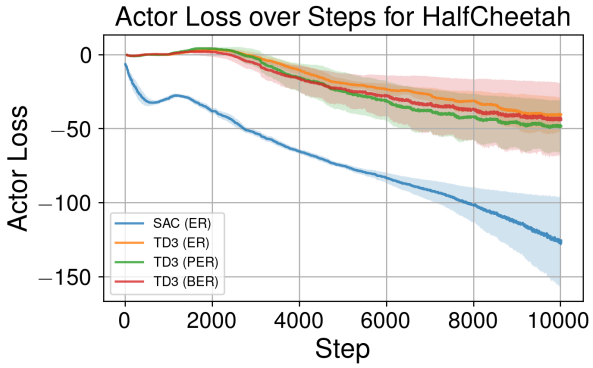

Figure 13: Actor Loss over the environment steps

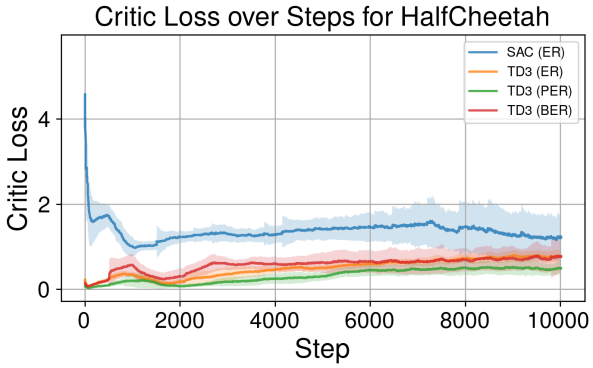

Figure 14: Critic Loss over the environment steps

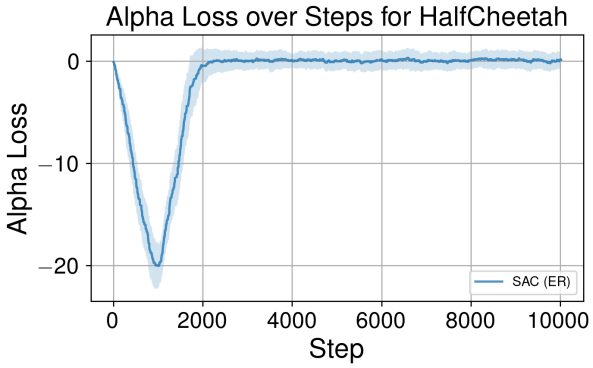

Figure 15: Alpha Loss over the environment steps

B.5 Hockey Evaluation Plots

In this section, we present the evaluation plots for the Hockey environment. The plots illustrate the performance of the TD3 and SAC algorithms over the environment steps. As observed, SAC converges faster than TD3 in terms of reward (Figure 16). However, PER converges at a local optimum rather than a global one, as seen in both reward and actor loss (Figure 17). The loss of own reward in all variants is the smallest for the actor. Additionally, ER performs better and with less uncertainty than BER. The following figures provide a detailed comparison of the two algorithms' performance metrics, including critic loss (Figure 18) and alpha loss (Figure 19).

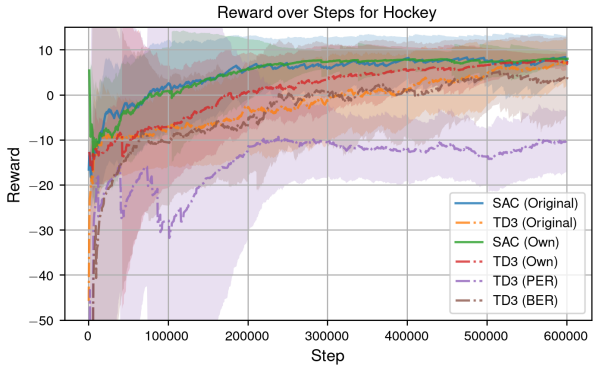

Figure 16: Reward over the environment steps

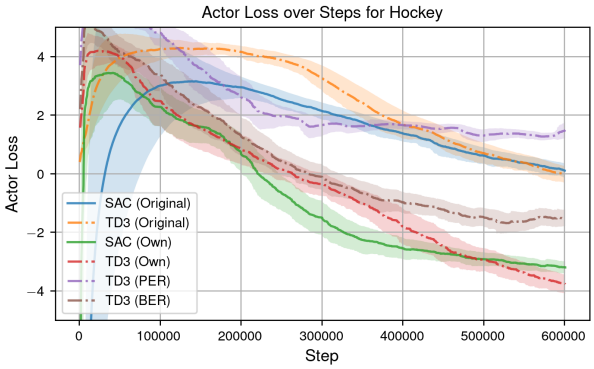

Figure 17: Actor Loss over the environment steps

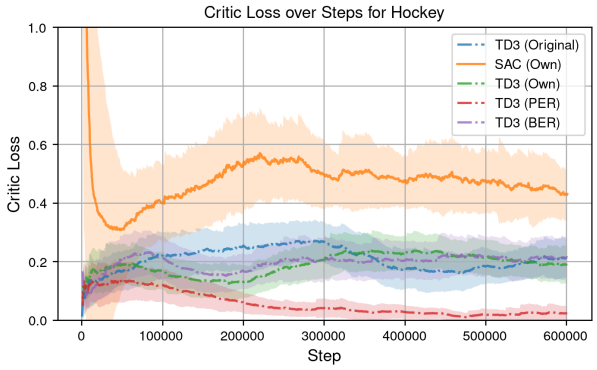

Figure 18: Critic Loss over the environment steps

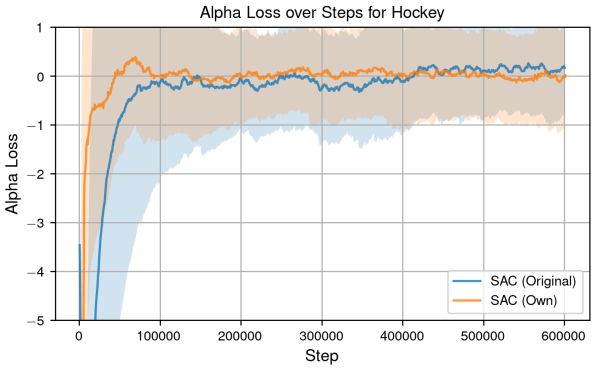

Figure 19: Alpha Loss over the environment steps

B.6 Simultaneous Self-Play Plots

Figure 20 shows how the evaluation reward of our three agents in simultaneous self-play. In this mode every 100 epochs (1000 steps) the agents and opponents are sampled from a predefined list of all agents to train. As result, the agents learn "online" to adapt to each other. With the figure follows that simultaneous self-play decreases the overall model performance (see also Table 6). Furthermore, the Hybrid model outperforms TD3 and SAC because it learns faster. As result, we descided to use fixed self-play that keeps the opponents fixed for the training of one agent. To ensure enough self-play, we use the past trained models as new opponents.

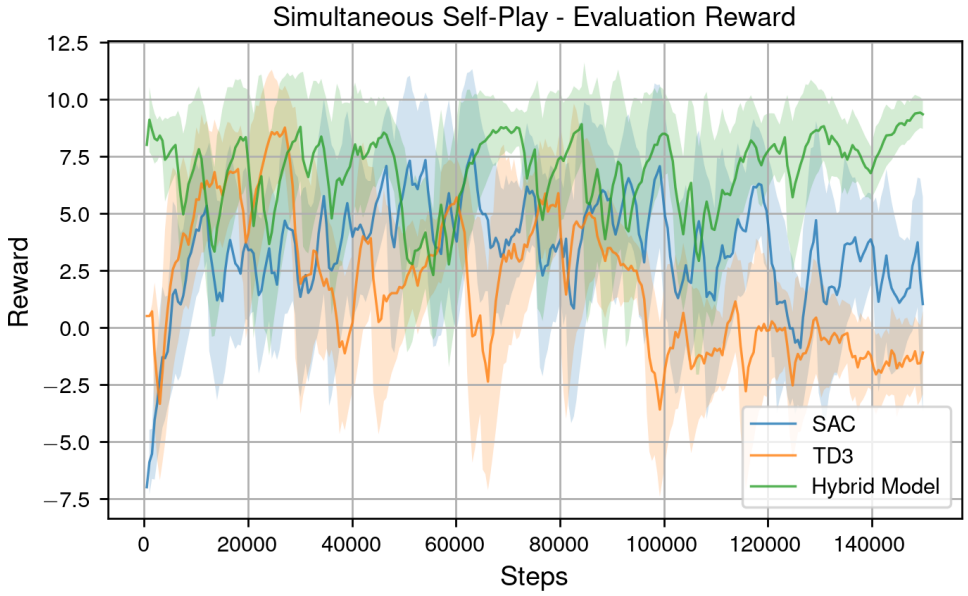

Figure 20: Selfplay Rewards Evaluation for SAC, TD3 and the Hybrid model.

B.7 Final Tournament

Table 6 contains our final tournament with the each tested approach.The tournament was done over 5000 matches with each 100 sub-games and leaderboard of our framework ranks agents dynamically based on their performance using the Placket-Luce rating system [8]. There, each agent has a skill rating average μ and variance σ, which are updated after every match. Games are simulated, and winners gain rating points while losers may lose them. Agents are paired based on a probability distribution over their skill levels, ensuring fair matchups. To ensure a diverse tournament, we first sample 4 agents and assign them probabilities based on the softmax of their σ. Based on them, the final 2 agents are sampled. Agent names, types, ratings, and average scores are displayed on the leaderboard. As we can see, the self-play agents result in worse ratings. We assume that our self-play mechanism causes instabilities. Thus, we changed to fixed self-play that results in our best agents.

Table 6: Leaderboard of the tournament

Rank	Player	Type	Rating	μ	σ
1	Hybrid Agent	Hybrid Agent	46.16	48.20	2.04
2	TD3 (Own)	TD3	36.82	38.64	1.81
3	TD3 (Original)	TD3	36.70	38.54	1.84
4	TD3 (White)	TD3	35.60	37.38	1.78
5	TD3 (Blue)	TD3	33.07	34.85	1.78
6	TD3 (BER)	TD3	32.51	34.26	1.76
7	SAC (Original)	SAC	31.06	32.78	1.73
8	SAC (Own)	SAC	30.07	31.81	1.73
9	Hybrid Agent (Selfplay)	Hybrid Agent	26.36	28.10	1.73
10	Hybrid Agent (Selfplay)	Hybrid Agent	24.12	25.84	1.72
11	SAC (Fine-Tuned)	SAC	22.10	23.79	1.69
12	Hybrid Agent (Expert Pretrain)	Hybrid Agent	21.72	23.43	1.71
13	Hybrid Agent (v3)	Hybrid Agent	20.26	21.96	1.71
14	SAC (Selfplay)	SAC	20.03	21.75	1.72
15	TD3 (PER)	TD3	19.04	20.73	1.69
16	Hybrid Agent	Hybrid Agent	18.72	20.42	1.70
17	TD3 (Foundation)	TD3	16.58	18.30	1.72
18	SAC (Foundation)	SAC	15.75	17.45	1.71
19	TD3 (Selfplay)	TD3	13.58	15.36	1.78
20	SAC (Selfplay)	SAC	13.24	15.02	1.78
21	TD3 (Selfplay)	TD3	10.99	12.78	1.79
22	Basic Opponent (Weak)	Basic Opponent	8.61	10.42	1.80
23	Basic Opponent (Strong)	Basic Opponent	7.81	9.66	1.84

B.8 Final Win-Rates

Table 7 contains the rewards of our best TD3, SAC and Hybrid agents based on the results of Table 6. Therefore, each model hast played 1000 games against each opponent to have enough statistics for our final evaluation.

Table 7: Performance between the agents and the baseline opponents.

Agent Opponent	Hybrid Model	SAC (Own)	TD3 (Own)
strong	94.60	97.10	**97.90**
weak	97.70	98.40	**99.20**